

Contents

1	JoBot Master Plan	7
1.1	Agentic Development, Refactor, Production Hardening, Feature Integration and Release Readiness Specification	7
2	0. How the implementation agent must use this document	7
2.1	0.1 Core operating invariants	8
2.2	0.2 North-star execution loop	9
3	1. Source coverage, authority and priority	9
3.1	1.1 Synthesis rule	9
3.2	1.2 Master priority assignment	10
3.3	1.3 Missing-source limitation	11
4	2. Repository truth that must shape implementation	11
4.1	2.1 Current architecture strengths	11
4.2	2.2 Current correctness gaps found directly in code	11
4.2.1	Durable task graph is currently not durable	11
4.2.2	Application pipeline has an architectural split between declared and enforced policy	12
4.2.3	Current pipeline-level idempotency is necessary but insufficient	12
4.2.4	Verification needs to become a real reconciliation protocol	12
4.2.5	Database migrations are currently additive and implicit .	12
4.2.6	LLM streaming is still stubbed	12
4.2.7	Tauri shell permissions are broader than necessary	12
4.2.8	Version and packaging metadata drift exists	12
4.2.9	Documentation is stale relative to the actual project state	13
4.2.10	Policy defaults need redesign, not just more code	13
4.3	2.3 Repository hygiene findings	13
5	3. Final product definition	13
5.1	3.1 Product promise	13
5.2	3.2 User-visible capabilities	14
5.3	3.3 Autonomy model	14
6	4. Target architecture	15
6.1	4.1 Logical package target	15
7	5. Phase 0 - Baseline, freeze and source reconciliation	17
7.1	Objective	17
7.2	Actions	17
7.2.1	5.1 Read and freeze contracts	17
7.2.2	5.2 Establish a baseline report	18
7.2.3	5.3 Freeze public contracts	18
7.2.4	5.4 Create a living requirements matrix	18

7.3	Exit criteria	19
8	6. Phase 1 - Release-blocking security and supply-chain hard- ening	19
8.1	6.1 Dependency and npm remediation	19
8.2	6.2 CodeQL URL sanitization	19
8.3	6.3 Vault hardening	20
8.4	6.4 CI security hardening	20
8.5	6.5 Tauri security hardening	20
8.6	6.6 Governance files	21
8.7	Exit criteria	21
9	7. Phase 2 - Durable execution core	21
9.1	7.1 Persistent task entities	21
9.2	7.2 Atomic task claiming	22
9.3	7.3 Event ledger	22
9.4	7.4 Event bus	23
9.5	7.5 Checkpointing	23
9.6	7.6 Retry policy	24
9.7	Exit criteria	24
10	8. Phase 3 - Application state machine, effects, approval and reconciliation	24
10.1	8.1 Application state machine	24
10.2	8.2 External-effect ledger	25
10.3	8.3 Durable approvals	25
10.4	8.4 Policy as an execution boundary	26
10.5	8.5 Risk model	26
10.6	8.6 Correct the response timestamp semantics	27
10.7	8.7 Reconciliation service	27
10.8	Exit criteria	27
11	9. Phase 4 - Browser reliability and adapter platform	27
11.1	9.1 Browser subsystem	27
11.2	9.2 Deterministic browser actions	28
11.3	9.3 Selector registry	28
11.4	9.4 Browser evidence	28
11.5	9.5 Site health	29
11.6	9.6 Platform terms and safety boundary	29
11.7	9.7 Adapter family generalization	29
11.8	9.8 Adapter boundary schemas	29
11.9	9.9 Direct ATS API integration - corrected policy	30
11.10	Exit criteria	30
12	10. Phase 5 - LLM platform, streaming, structured output and economics	30

12.1	10.1	Provider interface	30
12.2	10.2	Streaming UX	31
12.3	10.3	Capability-aware routing	31
12.4	10.4	Durable model economics	31
12.5	10.5	Context management	32
12.6		Exit criteria	32
13	11.	Phase 6 - Candidate truth, memory, prompts and evidence	32
13.1	11.1	Candidate Truth System	32
13.2	11.2	Grounded application answering	33
13.3	11.3	Layered memory	33
13.4	11.4	Memory retention and privacy	34
13.5	11.5	Prompt registry	34
13.6	11.6	Evidence system	34
13.7	11.7	Resume/cover pipeline	35
14	12.	Phase 7 - Discovery, matching, fraud checks and career intelligence	35
14.1	12.1	Multi-stage job matching	35
14.2	12.2	Job quality and fraud detection	36
14.3	12.3	Company normalization	36
14.4	12.4	Proactive job discovery	37
14.5	12.5	Application outcome learning	37
14.6	12.6	Career intelligence graph	37
14.7	12.7	Networking and referral tracking	38
15	13.	Phase 8 - GUI, TUI, control plane and reports	38
15.1	13.1	GUI is the control plane	38
	15.1.1	Home	38
	15.1.2	Task inspector	39
	15.1.3	Application detail	39
	15.1.4	Approval inbox	39
	15.1.5	Incident console	40
15.2	13.2	GUI feature backlog	40
15.3	13.3	TUI	40
15.4	13.4	HTML reports	41
16	14.	Phase 9 - CLI and repository-wide refactor	41
16.1	14.1	CLI decomposition	41
16.2	14.2	Core service boundaries	42
16.3	14.3	Adapter isolation	42
16.4	14.4	Database refactor	42
16.5	14.5	Error model	42
16.6	14.6	Structural refactor strategy	43
	16.6.1	Track A - low-regression refactor	43
	16.6.2	Track B - deep restructure	43

17	15. Phase 10 - MCP, API server, plugins and integrations	43
17.1	15.1 MCP as an interoperability boundary	43
17.2	15.2 API server mode	44
17.3	15.3 Plugin architecture	44
17.4	15.4 Browser extension	45
18	16. Phase 11 - Production packaging, release engineering and operations	45
18.1	16.1 One version authority	45
18.2	16.2 Python distribution	46
18.3	16.3 Docker	46
18.4	16.4 Desktop	46
18.5	16.5 Release channels	46
18.6	16.6 Release pipeline	47
18.7	16.7 Release gates	47
	18.7.1 Functional	47
	18.7.2 Reliability	47
	18.7.3 Security	48
	18.7.4 Packaging	48
18.8	16.8 jobot doctor	48
18.9	16.9 Backup and recovery	48
19	17. Phase 12 - Observability, privacy and governance	49
19.1	17.1 OpenTelemetry	49
19.2	17.2 Structured logging	50
19.3	17.3 Audit log	50
19.4	17.4 Telemetry privacy	50
19.5	17.5 Privacy documentation	50
20	18. Phase 13 - Evaluation platform and agent reliability science	51
20.1	18.1 Capability evals	51
20.2	18.2 Reliability evals	51
20.3	18.3 Security evals	51
20.4	18.4 Long-horizon evals	52
20.5	18.5 Failure injection	52
20.6	18.6 Regression policy	53
21	19. Phase 14 - Self-improvement, skill extraction and bounded autonomy	53
21.1	19.1 Capability acquisition loop	53
21.2	19.2 Skill extraction	53
21.3	19.3 Bounded self-improvement	54
21.4	19.4 Trust promotion/demotion	54
22	20. Phase 15 - Data model and migration overhaul	54
22.1	20.1 Versioned migrations	54

22.2	20.2 Repository layer	55
22.3	20.3 Indexing strategy	55
22.4	20.4 SQLite durability settings	55
23	21. Phase 16 - Performance and resource-efficiency program	56
23.1	21.1 Baseline before optimization	56
23.2	21.2 Python efficiency	57
23.3	21.3 Browser efficiency	57
23.4	21.4 LLM efficiency	57
23.5	21.5 Storage efficiency	57
23.6	21.6 GUI efficiency	58
23.7	21.7 Sidecar efficiency	58
23.8	21.8 Performance gates	58
24	22. Phase 17 - Repository cleanup and unnecessary-file removal	58
24.1	22.1 Canonical top-level structure	59
24.2	22.2 Documentation consolidation	59
24.3	22.3 Duplicate detection	60
24.4	22.4 Generated-file cleanup	60
24.5	22.5 Secret-like file audit	60
24.6	22.6 Plan/document authority	61
24.7	Exit criteria	61
25	23. Phase 18 - Documentation generation program	61
25.1	23.1 Required documentation set	61
	25.1.1 User documentation	61
	25.1.2 Developer documentation	61
	25.1.3 Operations documentation	62
25.2	23.2 Generated references	62
25.3	23.3 Documentation verification	62
26	24. Feature integration matrix - complete unified backlog	63
27	25. Competitive feature synthesis	66
28	26. P0 backlog - release-blocking work	67
28.1	Correctness	67
28.2	Security	67
28.3	Verification	68
28.4	Release	68
29	27. P1 backlog - v1.x product expansion	68
30	28. P2 strategic moat	69
31	29. First production milestone - Durable Verified Application	69

32 30. Release-readiness definition of done	70
32.1 Product	70
32.2 Architecture	70
32.3 Security	71
32.4 Reliability	71
32.5 Observability	71
32.6 Packaging	71
32.7 Documentation	72
32.8 Governance	72
32.9 Evidence	72
33 31. Agent execution protocol	72
33.1 31.1 Before every task	73
33.2 31.2 During implementation	73
33.3 31.3 After implementation	73
33.4 31.4 Worktree isolation	73
33.5 31.5 Worker timeouts and heartbeats	74
33.6 31.6 Approval boundary	74
33.7 31.7 No fake completion	74
34 32. Suggested workstream decomposition	74
34.1 WS-A - Durable core	74
34.2 WS-B - Security and release	75
34.3 WS-C - Application reliability	75
34.4 WS-D - AI and documents	75
34.5 WS-E - Product surface	75
34.6 WS-F - Discovery and career intelligence	75
34.7 WS-G - Packaging and launch	76
35 33. High-risk implementation items requiring special handling	76
35.1 LinkedIn	76
35.2 Direct ATS APIs	76
35.3 Credential-bearing tools	76
35.4 Generated candidate content	76
35.5 Self-improvement	76
35.6 Telemetry	77
35.7 Multi-worker execution	77
36 34. Documentation and artifact checklist for the final implementation agent	77
37 35. Final execution sequence	78
38 36. Recommended release sequence	79
39 37. Post-launch operating model	79
39.1 Weekly	79

39.2 Monthly	79
39.3 Per release	80
39.4 Continuous	80
40 38. Final design principles	80
41 39. External research basis	81
42 40. Master completion rule	81

1 JoBot Master Plan

1.1 Agentic Development, Refactor, Production Hardening, Feature Integration and Release Readiness Specification

Document type: Authoritative implementation plan for an AI coding/development agent **Target repository:** <https://github.com/aryansinghnagar/JoBot>
Target product: JoBot - a local-first, privacy-preserving, human-governed autonomous job-application operating system **Release target:** v1.0.0 production release for an average user, with an architecture capable of later expansion into a broader agent operating system **Primary source of operating principles:** repository AGENTS.md **Source plans synthesized:** Plan1.md, Plan1.pdf, Plan2.md, Plan3.md, plan4.md, plan5.md **Unavailable at synthesis time:** plan6.md, plan7.md, plan8.md, plan9.md, plan10.md, plan8.pdf **Status:** Master synthesis - implementation-ready, subject to the source-availability limitation above

2 0. How the implementation agent must use this document

This document is not a wish list. Treat it as an execution specification that sits immediately below AGENTS.md in authority and above individual implementation tasks.

The implementation agent MUST:

1. Read AGENTS.md completely before changing code.
2. Inspect the current repository before trusting any statement in this plan. Repository truth outranks stale plan text.
3. Treat this plan as the unified intent of the available plans, not as five independent checklists.
4. Preserve working behavior unless a change is explicitly justified by correctness, security, maintainability, performance, or product requirements.

5. Prefer incremental migrations over rewrites.
6. Make state durable before making autonomy broad.
7. Never equate model output, browser completion, or a local status flag with verified real-world success.
8. Never execute an external side effect merely because it is possible. Policy, risk, approval, idempotency, and verification are mandatory controls.
9. Never bypass or defeat platform security, access controls, CAPTCHAs, rate limits, or explicit terms of service. For automation that is not clearly authorized, default to assisted/manual flows and clearly surface the limitation to the user.
10. Work in small, reversible changes. After each meaningful change, run the smallest relevant test/eval loop, record evidence, and then continue.
11. Keep `queues/now.md`, `queues/next.md`, `queues/blocked.md`, `queues/improve.md`, `worklog.md`, `status.md`, `decisions.md`, and `FAILURE.md` synchronized with actual repository state.
12. End every phase with an observable artifact and verification result.
13. Never mark a feature complete because code exists. A feature is complete only when contract, tests, failure handling, evidence, documentation, and release behavior are all present.
14. When a decision is not material and the plan provides a safe default, choose the default and proceed. Do not interrupt execution with low-value clarification questions.
15. Ask a concise clarification only when an ambiguity can materially change architecture, legal/compliance posture, release scope, or user data safety, and the answer cannot be inferred conservatively.
16. When an implementation reveals a better architecture than this plan, preserve the intent and improve the design. Do not blindly follow stale implementation details.
17. Do not leave temporary scaffolding, duplicate documentation, fake adapters, dead code, or unverified claims behind.

2.1 0.1 Core operating invariants

The implementation must preserve and eventually enforce these invariants at runtime:

NO ACTION WITHOUT DURABLE STATE
 NO STATE CHANGE WITHOUT AN EVENT
 NO COMPLETION WITHOUT VERIFICATION
 NO CONSEQUENTIAL SIDE EFFECT WITHOUT POLICY
 NO RETRY WITHOUT IDEMPOTENCY
 NO LONG-RUNNING WORK WITHOUT CHECKPOINTS
 NO MEMORY WITHOUT PROVENANCE
 NO AUTONOMY WITHOUT MEASUREMENT
 NO PLUGIN WITHOUT PERMISSIONS
 NO SECRET IN LOGS, PROMPTS, EVENTS OR TELEMETRY

NO UNKNOWN STATE TREATED AS SUCCESS
NO RELEASE WITHOUT REPRODUCIBLE EVIDENCE

2.2 0.2 North-star execution loop

Every substantial JoBot workflow must converge toward:

User goal

- > explicit plan/task graph
- > durable execution
- > tool/adaptor action
- > policy/risk decision
- > verification
- > evidence
- > state update
- > memory update
- > metrics
- > learning candidate
- > evaluation
- > bounded improvement

The core product is therefore not a chat interface and not a mass-apply loop. It is a durable execution system specialized for the career domain.

3 1. Source coverage, authority and priority

3.1 1.1 Synthesis rule

The source plans overlap heavily. Do not implement them sequentially as five separate programs. Treat them as different lenses over one system:

- **Plan1.md**: architectural spine, durable execution, verification, memory, security boundaries, evidence, evaluation, release gates, and long-term career-OS direction.
- **Plan1.pdf**: corroborating/rendered form of Plan 1. It is treated as a semantic duplicate unless an exact content comparison reveals unique material.
- **Plan2.md**: product completeness, competitive feature parity, LLM streaming, CLI refactor, LinkedIn/application flows, GUI, telemetry, packaging, and differentiated features.
- **Plan3.md**: incremental refactor strategy, adapter-family generalization, selector healing, unwired subsystem completion, boundary schemas, event bus, plugins, MCP, async hot paths, and GUI-first feature backlog.
- **plan4.md**: release engineering, reproducible packaging, desktop distribution, observability/privacy, documentation site, community governance,

launch operations.

- **plan5.md**: release-blocking security and supply-chain remediation, dependency upgrades, CodeQL URL sanitization, CI hardening, Tauri security, vault hardening, packaging metadata, and repository hygiene.

3.2 1.2 Master priority assignment

The names of the plans are not used to determine execution order. Priority is assigned by consequence, dependency, risk, and leverage.

Source	Master priority	Why
Plan 1	P0 backbone	Defines the execution model, correctness invariants, durable runtime, evidence, evaluation, and release gates.
Plan 5	P0 release blocker	Contains concrete security defects that can invalidate an otherwise functional release.
plan4	P0 release track	Converts the project into a reproducible distributable product with governance and operational controls.
Plan 3	P0-P1 refactor/extension	Provides incremental restructuring and feature-family generalization that fits the architecture.
Plan 2	P1 product expansion	Provides market parity and user-facing capability expansion after the substrate is safe.
Plan 1.pdf	Evidence duplicate	Preserve as source evidence if needed, but do not create duplicated implementation work.

3.3 1.3 Missing-source limitation

The requested `plan6.md` through `plan10.md` and `plan8.pdf` were not available in the accessible conversation/library sources during this synthesis. They are therefore not represented as if they were read. The implementation agent **MUST** perform a source reconciliation pass before execution if those files become available later. New ideas from those documents should be merged through the same dependency and conflict-resolution rules in this plan rather than appended as a second roadmap.

4 2. Repository truth that must shape implementation

A live audit of the repository reveals several important facts that should override stale descriptions in older plans.

4.1 2.1 Current architecture strengths

JoBot already contains substantial production-oriented building blocks:

- Python 3.11+ core.
- SQLite WAL persistence and encrypted local secrets.
- Provider-neutral model routing and cost awareness.
- A 12-phase application submission pipeline with DoD gates.
- Site adapter abstractions for multiple ATS/job sources.
- Circuit breakers and verification hooks.
- Tauri 2 + React GUI shell and a JSON-RPC sidecar.
- Resume tailoring, cover letters, ATS checking and PDF generation.
- Job discovery, deduplication and JobSpy integration.
- PII masking, vault/keyring integration, policy controls, scheduling, notifications, tracker/analytics, mock ATS fixtures and a broad automated test suite.
- GitHub Actions, CodeQL, Dependabot, SBOM/provenance, Docker packaging and PyPI publishing infrastructure.
- Project state/queue files that already align with the `AGENTS.md` filesystem-first model.

4.2 2.2 Current correctness gaps found directly in code

4.2.1 Durable task graph is currently not durable

`src/jobot/task_graph.py` stores tasks in an in-memory dictionary. Task claiming mutates in-memory objects and therefore is not an atomic database-backed lease protocol. The implementation contract must move task ownership, attempts, leases, heartbeats, recovery and task events into durable storage.

4.2.2 Application pipeline has an architectural split between declared and enforced policy

`ApplicationSubmissionPipeline` has a `PolicyEngine`, but the current phase execution path does not make policy evaluation the universal pre-side-effect gate. Phase 10 can pause on trust level, while phase 11 performs the external submit operation. The refactor must make policy/risk evaluation mandatory immediately before every consequential side effect, not merely an injected dependency.

4.2.3 Current pipeline-level idempotency is necessary but insufficient

The pipeline derives an idempotency key from `job_url` and `profile_id`, detects an existing application, and avoids some duplicate behavior. This must be extended to an external-effect ledger so ambiguous local state can never trigger a duplicate remote submission.

4.2.4 Verification needs to become a real reconciliation protocol

The current phase 12 verification marks an application `VERIFIED` when adapter verification succeeds, but the master runtime needs a first-class `UNKNOWN` state, evidence correlation, confirmation identifiers, replay protection, and recovery behavior for uncertain outcomes.

4.2.5 Database migrations are currently additive and implicit

`DatabaseManager.migrate()` uses ad-hoc `_ensure_column()` logic. That is adequate for early development but not for a long-lived product with upgrade, rollback and recovery guarantees. Introduce checksummed, versioned migrations.

4.2.6 LLM streaming is still stubbed

At least several provider classes still raise `NotImplementedError` for streaming. This directly blocks real-time UX and efficient long-form workflows. Streaming must be implemented behind one provider-independent interface, with tests for all provider families and a graceful fallback to non-streaming.

4.2.7 Tauri shell permissions are broader than necessary

The current capability configuration gives the sidecar `spawn/execute` permissions with unrestricted argument matching (`args: true`). This must be replaced with least-privilege command/argument allowlists.

4.2.8 Version and packaging metadata drift exists

The current `pyproject.toml` declares version 0.1.0, supports Python `>=3.11`, and still uses the deprecated table-form license declaration while the

GUI/release plans target a different release version. Establish one authoritative version source and make all packaging metadata derive from it.

4.2.9 Documentation is stale relative to the actual project state

The current README says the authoritative specification is an older merge plan and still describes Patchright integration as being in progress. This is now incompatible with a release-ready product narrative and must be replaced by generated, tested documentation that reflects reality.

4.2.10 Policy defaults need redesign, not just more code

The current `PolicyEngine` includes large site-specific daily application limits. The future risk model must separate recommendation limits, preparation limits, submission limits, per-site trust, per-action risk, and user-defined caps. High numerical caps must not be mistaken for production-safe autonomy.

4.3 2.3 Repository hygiene findings

The repository currently contains multiple planning generations and very large historical documents alongside the active product. Examples visible in the tree include large historical planning files, root plan files, `repo_research.md`, `worklog.md`, multiple implementation contracts, and old merged plans. The cleanup phase must establish one canonical documentation hierarchy, archive historical material, and delete only files proven redundant.

The cleanup must be evidence-driven: classify every tracked file as active runtime, active test, active documentation, generated artifact, historical record, duplicate, obsolete, secret-like, or unknown. Unknown files are investigated before deletion.

5 3. Final product definition

5.1 3.1 Product promise

For v1.0, JoBot should truthfully promise:

JoBot discovers, evaluates, prepares, verifies, submits where technically and policy-authorized, and tracks job applications across a defined set of supported sources and workflows. It provides durable state, human-governed automation, crash recovery, evidence, auditable execution, privacy-preserving local operation, and release-gated evaluation.

Do not promise universal job-board support, unrestricted autonomy, guaranteed interview outcomes, or platform-safe automation where the underlying platform

does not authorize it.

5.2 3.2 User-visible capabilities

At release, an average user should be able to:

1. Install JoBot from a documented distribution channel.
2. Run `jobot doctor` and receive actionable environment status.
3. Import an existing resume and create a secure candidate profile.
4. Configure LLM providers and privacy settings without editing source code.
5. Discover jobs across supported sources.
6. See normalized jobs and match explanations.
7. Tailor a resume and cover letter using grounded candidate facts.
8. Review form questions and receive profile-grounded answers.
9. Prepare an application without committing an external side effect.
10. Approve, edit, reject or defer consequential steps.
11. Submit through supported and authorized mechanisms.
12. Verify submission using independent evidence.
13. Recover after process, browser, network or provider failures.
14. Track applications, interviews, follow-ups, contacts and outcomes.
15. Inspect cost, evidence, logs, incidents and decisions.
16. Export their data and purge it locally.
17. Upgrade from one release to the next without losing state.

5.3 3.3 Autonomy model

Autonomy must be scoped, explicit and measurable.

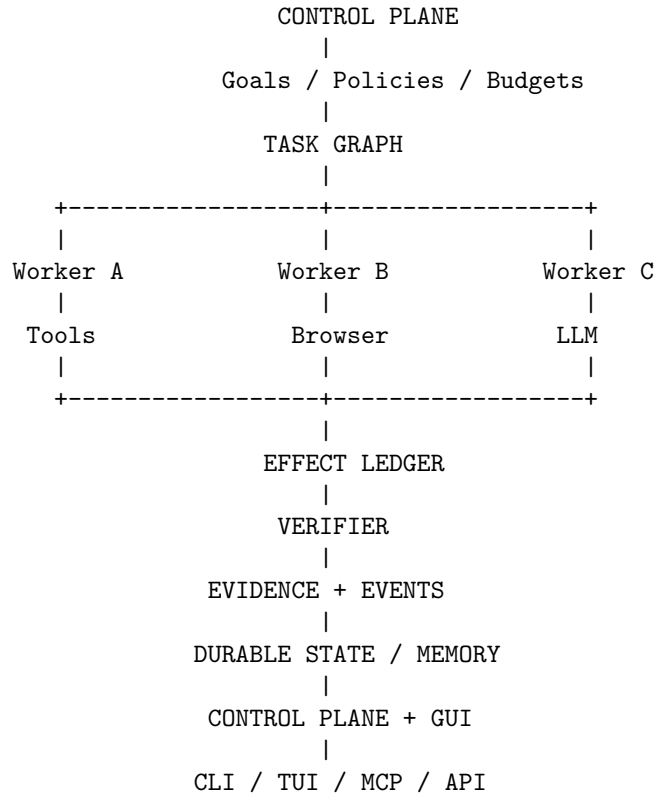
Risk	Example	Default behavior
R0	Local read/inspect	Automatic
R1	Public job discovery	Automatic
R2	Resume/cover generation	Automatic, with verification
R3	Application preparation	Automatic, approval before external side effect
R4	Save/submit action on a trusted authorized integration	Policy-controlled
R5	Final job submission on a browser site	Human approval by default
R6	Recruiter outreach / external messaging	Human approval
R7	Credential/secret modification	Human approval

Risk	Example	Default behavior
R8	Irreversible/high-impact action	Human only

Trust must be scoped to the target, action type, integration, skill or adapter. There is no single global `autonomous=true` switch.

6 4. Target architecture

The architecture should converge incrementally toward the following layers.



6.1 4.1 Logical package target

Do not perform a single giant directory rewrite. Converge over time toward:

```
src/jobot/
  core/
```

- events/
- state/
- tasks/
- workflows/
- errors/
- control/
 - goals/
 - approvals/
 - budgets/
 - trust/
 - policies/
 - incidents/
- execution/
 - workers/
 - leases/
 - sandbox/
 - browser/
 - tools/
- ai/
 - router/
 - providers/
 - prompts/
 - profiles/
 - evaluation/
- memory/
 - semantic/
 - episodic/
 - procedural/
 - retrieval/
- career/
 - matching/
 - scoring/
 - market/
 - networking/
 - interview/
- applications/
 - state_machine/
 - preparation/
 - submission/
 - verification/
- adapters/
 - ats/
 - boards/
 - browser/
- documents/
 - resume/


```
cover_letter/  
pdf/  
ats/  
observability/  
events/  
tracing/  
metrics/  
incidents/  
plugins/
```

Keep compatibility shims while moving modules.

7 5. Phase 0 - Baseline, freeze and source reconciliation

7.1 Objective

Establish a reproducible starting point and prevent plan/code drift from contaminating implementation.

7.2 Actions

7.2.1 5.1 Read and freeze contracts

Read and reconcile:

- AGENTS.md
- .agents/agents.md
- SETUP.md
- docs/contracts.md
- existing plan.md
- worklog.md
- queues/*.md
- current CI workflows
- packaging files
- GUI sidecar contract
- current tests

Create:

- docs/implementation/master-plan.md as the canonical checked-in version of this document.
- docs/implementation/requirements-matrix.md mapping every requirement to implementation location and verification.
- docs/implementation/source-reconciliation.md recording all source plans present/missing and their deduplicated ideas.

7.2.2 5.2 Establish a baseline report

Capture:

- Python and Node versions.
- OS.
- dependency lock state.
- test count and pass/fail/skip count.
- lint/type status.
- GUI build status.
- package build status.
- Docker smoke status.
- database schema version.
- current application pipeline status.
- current adapter inventory and live/mock status.
- current code-size hotspots.
- current cold-start time, memory footprint, CPU usage and representative workflow timings.
- current security scan outputs.

7.2.3 5.3 Freeze public contracts

Before refactoring, define regression tests for:

- CLI command names and important options.
- public Python interfaces used by adapters and plugins.
- JSON-RPC sidecar method names and request/response schemas.
- database-compatible serialization.
- profile import/export formats.
- job/application status enumerations.
- artifact directory conventions.

7.2.4 5.4 Create a living requirements matrix

Every row MUST include:

Requirement ID
Source plan(s)
Priority
Feature/component
Current state
Implementation target
Dependencies
Tests/evals
Observability
Docs
Security concerns
Performance budget

Release gate
Status

7.3 Exit criteria

- Baseline captured.
 - No stale claims remain in the implementation matrix.
 - Tests and gates have a known baseline.
 - Every available source plan is represented exactly once in the matrix.
-

8 6. Phase 1 - Release-blocking security and supply-chain hardening

This phase is P0 and must happen before release-facing feature expansion.

8.1 6.1 Dependency and npm remediation

Implement the current dependency hardening from the source plans, but do not blindly pin to versions mentioned in historical plans. Resolve to versions that are current and compatible at implementation time.

Actions:

- upgrade Vite/Vitest/Tauri/React toolchain as required by current security advisories and compatibility.
- ensure Node support policy is explicit.
- use dependency overrides only when necessary and documented.
- run `npm audit` at a release-appropriate severity gate.
- verify no transitive vulnerable packages remain without documented residual risk.

8.2 6.2 CodeQL URL sanitization

Replace substring-based site inference with structured URL parsing.

Required behavior:

- normalize scheme/host/port/userinfo safely.
- exact host or valid subdomain matching only.
- reject crafted hostnames such as `greenhouse.io.attacker.com`.
- reject query-string tricks such as a malicious URL containing `lever.co` only in its query.
- reject unknown hosts rather than silently falling back to an adapter.
- preserve user-friendly errors through CLI, GUI and sidecar layers.

Add adversarial tests covering:

- uppercase hosts.
- explicit/non-default ports.
- userinfo fields.
- trailing dots/slashes.
- scheme-less inputs.
- IDN/punycode edge cases.
- nested subdomains.
- attacker-controlled suffixes.

8.3 6.3 Vault hardening

Implement:

- atomic 0600 key creation.
- ownership and permission checks.
- `0_NOFOLLOW` where supported.
- Windows ACL expectations documented and tested where feasible.
- no symlink-follow writes.
- key rotation strategy.
- test cases for world-readable keys and symlink attacks.

8.4 6.4 CI security hardening

Add or enforce:

- `pip-audit`.
- `npm audit`.
- `gitleaks` over full history.
- CodeQL coverage for Python, JavaScript/TypeScript and Rust where applicable.
- `actionlint`.
- SHA-pinned GitHub Actions.
- least-privilege workflow permissions.
- Dependabot coverage for Python, npm and Cargo.
- grouped update strategy for low-risk upgrades.
- import-smoke test with base installation.
- optional `cargo audit` / `cargo deny` in desktop CI.

8.5 6.5 Tauri security hardening

Implement:

- restrictive CSP.
- explicit capabilities.
- no unrestricted shell argument matching.
- explicit window labels.
- least-privilege IPC.

- audited sidcar commands.
- updater signing keys and verification.
- honest code-signing status per OS.

Tauri’s current security model explicitly treats capabilities as the mechanism for limiting frontend exposure to system interfaces, and recommends tightly scoped capability files. CSP should be configured as restrictively as practical.

8.6 6.6 Governance files

Create/update:

- SECURITY.md
- CONTRIBUTING.md
- CODE_OF_CONDUCT.md
- CODEOWNERS
- issue templates
- pull request template
- FUNDING.yml
- .editorconfig
- license/copyright metadata

8.7 Exit criteria

- All release-blocking security findings resolved or explicitly accepted with documented mitigation and owner.
- No secret scan findings.
- URL-inference adversarial suite green.
- Tauri capability audit complete.
- Security documentation matches actual implementation.

9 7. Phase 2 - Durable execution core

This phase implements the central architectural recommendation from Plan 1 and aligns the repository with AGENTS.md.

9.1 7.1 Persistent task entities

Introduce durable entities:

Goal
Task
TaskDependency
TaskAttempt
TaskLease

TaskEvent
TaskArtifact

Task states:

PENDING
READY
CLAIMED
RUNNING
WAITING
RETRYING
VERIFYING
COMPLETED
FAILED
QUARANTINED
CANCELLED
UNKNOWN

9.2 7.2 Atomic task claiming

Claiming MUST be a database transaction, not an in-memory mutation.

Required semantics:

- worker identifier.
- lease acquisition time.
- lease expiry.
- heartbeat.
- attempt number.
- strategy used.
- last checkpoint.
- failure reason.
- recovery eligibility.
- cancellation token.

Two workers must never hold the same active lease.

9.3 7.3 Event ledger

Create a durable append-only event table:

event_id
aggregate_type
aggregate_id
event_type
event_version
payload
actor
correlation_id

causation_id
created_at

Event examples:

GoalCreated
TaskCreated
TaskClaimed
TaskStarted
ToolInvoked
ToolCompleted
ToolFailed
ApprovalRequested
ApprovalGranted
ApprovalRejected
BrowserSessionStarted
BrowserSessionLost
ApplicationPrepared
ApplicationSubmitted
ApplicationVerified
ApplicationUnknown
BudgetExceeded
WorkerDisconnected
RunResumed
RunQuarantined
MemoryUpdated
EvalCompleted

9.4 7.4 Event bus

After the event ledger is stable, add a typed in-process event bus with subscribers for:

- alerts.
- telemetry.
- GUI push updates.
- analytics.
- persistence projections.
- incident handling.

Keep the event ledger as the source of truth; the event bus is a delivery mechanism, not the canonical state store.

9.5 7.5 Checkpointing

Every long-running harness must support:

- start checkpoint.

- intermediate checkpoint.
- approval waitpoint.
- tool result checkpoint.
- verification checkpoint.
- resume after process death.
- retry after recoverable failure.
- quarantine after unsafe ambiguity.

9.6 7.6 Retry policy

Default:

- retry once with the same strategy for transient failures.
- if it fails again, change strategy or escalate.
- never retry irreversible effects without reconciliation.
- never retry on hard policy denials.
- backoff must be explicit and bounded.

9.7 Exit criteria

Kill a worker at every major phase and demonstrate correct recovery without task duplication or lost durable state.

10 8. Phase 3 - Application state machine, effects, approval and reconciliation

10.1 8.1 Application state machine

Define explicit transitions for:

DISCOVERED
 NORMALIZED
 MATCHED
 SHORTLISTED
 PREPARING
 READY_FOR_REVIEW
 PENDING_APPROVAL
 SUBMITTING
 SUBMITTED
 VERIFYING
 VERIFIED
 UNKNOWN
 FAILED
 QUARANTINED
 CANCELLED

REJECTED
EXPIRED

Transitions must be validated against an explicit table. Do not allow arbitrary enum mutation.

10.2 8.2 External-effect ledger

Introduce:

```
ExternalEffect
  effect_id
task_id
application_id
effect_type
idempotency_key
request_hash
started_at
completed_at
status
external_reference
verification_state
compensation_state
```

Before an external side effect:

1. compute deterministic effect identity.
2. reserve the effect.
3. evaluate policy.
4. obtain approval where required.
5. execute once.
6. capture evidence.
7. reconcile remote state.
8. finalize ledger record.

If local state is ambiguous after a crash, reconcile instead of replaying the effect.

10.3 8.3 Durable approvals

Create persistent `ApprovalRequest` entities containing:

- proposed action.
- arguments.
- risk level.
- evidence.
- policy rationale.
- expiry.
- requester/actor.
- decision timestamp.

- edited arguments.
- resulting effect id.

Supported decisions:

APPROVE
EDIT
REJECT
DEFER
CANCEL

CLI, GUI and MCP must all use the same approval entity and lifecycle.

10.4 8.4 Policy as an execution boundary

Every consequential action must pass:

```
action proposal
-> policy
-> risk score
-> trust check
-> budget check
-> approval check
-> idempotency/effect reservation
-> execute
-> verify
```

The policy engine should return typed reasons and machine-readable decisions.

10.5 8.5 Risk model

Compute risk from:

```
action
target
reversibility
credential access
external side effect
personal data
financial cost
volume
confidence
integration trust
skill trust
```

Use policy rules for hard blocks and a risk score for escalation/approval routing.

10.6 8.6 Correct the response timestamp semantics

The current database code treats statuses such as `SUBMITTED` and `VERIFIED` as response statuses for `responded_at`. That conflates application submission with employer response. Split these concepts into:

- `submitted_at`.
- `submission_verified_at`.
- `first_employer_response_at`.
- `current_outcome`.

Add migration and backfill logic.

10.7 8.7 Reconciliation service

Implement reconciliation routines for:

- application submission.
- job expiry.
- provider status.
- email/application status signals.
- browser session ambiguity.
- remote job identity changes.

Unknown is not failure and not success. Unknown is a durable state requiring a next action.

10.8 Exit criteria

- Duplicate external submissions are provably prevented under crash/restart tests.
- Approval survives restart.
- A synthetic crash after submit but before local persistence is recoverable through reconciliation.

11 9. Phase 4 - Browser reliability and adapter platform

11.1 9.1 Browser subsystem

Create explicit services for:

`BrowserSessionManager`
`BrowserPool`
`ProfileStore`
`SessionPersistence`
`Navigation`

NamedActions
SelectorRegistry
SelectorHealing
EvidenceCapture
RateLimitDetector
SiteHealth
Recovery
CAPTCHA Boundary

11.2 9.2 Deterministic browser actions

Prefer semantic, stable locators and named actions. Playwright documents locators as the primary mechanism with built-in auto-waiting/retry behavior and actionability checks. JoBot should build its reliability layer on top of those deterministic primitives instead of encoding entire workflows as free-form model output.

11.3 9.3 Selector registry

Create a central registry:

site -> workflow -> step -> candidate locators

Support:

- multiple candidate selectors.
- confidence scoring.
- fallback order.
- healing records.
- DOM drift test fixtures.
- selector health metrics.

Do not allow silent AI rewriting of selectors in production. Proposed healing must pass bounded validation.

11.4 9.4 Browser evidence

For consequential actions capture:

- pre-action screenshot when useful.
- relevant DOM/form snapshot.
- named action and arguments.
- post-action screenshot.
- result.
- verification signal.
- trace/application ids.

Avoid storing massive raw browser state indefinitely. Apply retention and deduplication.

11.5 9.5 Site health

Track per integration:

- success rate.
- median latency.
- timeout rate.
- verification failures.
- selector drift.
- circuit breaker state.
- rate limit signals.
- recent user overrides.

Auto-demote unreliable integrations to supervised/manual mode.

11.6 9.6 Platform terms and safety boundary

LinkedIn currently prohibits unauthorized automated access and scraping in its User Agreement. Therefore, LinkedIn automation must never be implemented as a promise to bypass controls. Any browser automation work must include an explicit ToS/authorization review, user warning, rate controls, and a manual-assist fallback.

The same principle applies to every platform: do not defeat CAPTCHAs, security boundaries, anti-bot controls, access controls or usage limits. A CAPTCHA subsystem, if retained, should be a detection/escalation boundary rather than an unconditional bypass mechanism.

11.7 9.7 Adapter family generalization

Generalize common ATS patterns:

- Workday/CXS style APIs.
- Workable.
- Recruitee.
- Teamtailor.
- BambooHR.
- other compatible JSON feeds.

Use a shared transport/parser abstraction with per-site configuration and small site-specific modules.

11.8 9.8 Adapter boundary schemas

Define strict Pydantic contracts for:

- `JobPosting`.
- `ApplicationForm`.
- `ApplicationResult`.

- `SubmitOutcome`.
- `VerificationResult`.
- `SiteHealth`.

Invalid adapter output should fail safely and be quarantined.

11.9 9.9 Direct ATS API integration - corrected policy

Keep the idea of direct API submission, but implement it only where the necessary authorization is legitimately available.

For example, Greenhouse exposes public job-board reads but its application submission endpoint requires Basic Auth with an API key; Ashby's application form endpoint also requires authorization. These are not equivalent to unrestricted public applicant APIs.

Therefore, create three explicit adapter modes:

1. `PUBLIC_READ`: public discovery/normalization.
2. `USER_AUTHORIZED_API`: direct submission with valid user/employer-authorized credentials.
3. `BROWSER_ASSISTED`: hosted form/browser flow where direct API submission is not legitimately available.

Never fake an API flow merely because an endpoint exists.

11.10 Exit criteria

Mock ATS workflows survive injected browser, selector, timeout and network failures. Every supported production adapter documents its authorization mode and verification behavior.

12 10. Phase 5 - LLM platform, streaming, structured output and economics

12.1 10.1 Provider interface

Every provider must implement a common contract for:

- completion.
- streaming.
- structured output.
- tool calls where supported.
- token accounting.
- health checks.
- timeout behavior.
- error normalization.

- cancellation.

Implement streaming for the current provider families through provider-specific protocol adapters, then expose one `generate_text_stream()` interface in the router.

12.2 10.2 Streaming UX

Wire streaming to:

- `jobot qa`.
- `jobot coverletter`.
- `jobot interview`.
- conversational assistant.
- GUI generation views.

The non-streaming path must remain a fallback.

12.3 10.3 Capability-aware routing

Represent model requirements:

```
task: resume_tailoring
requirements:
  structured_output: true
  long_context: true
  reasoning: medium
  latency: medium
  cost: low
```

Route using:

- capability.
- quality score.
- latency.
- cost.
- provider health.
- historical task success.
- privacy requirements.
- context size.
- structured-output reliability.

12.4 10.4 Durable model economics

Create tables for:

- `llm_calls`.
- `budgets`.
- `budget_reservations`.

- `provider_health`.
- `model_capabilities`.
- `routing_decisions`.

Support:

- per-call estimated/actual cost.
- per-run cost.
- per-application cost.
- per-provider cost.
- daily/weekly/monthly budgets.
- reservation before expensive work.
- graceful fallback to cheaper/local models.

12.5 10.5 Context management

Implement:

- context snapshots.
- compact task summaries.
- bounded history.
- retrieval-first memory access.
- prompt caching where supported.
- tool result summarization.
- artifact references instead of copying huge files into context.
- automatic truncation with loss-of-critical-context tests.

12.6 Exit criteria

- Streaming works for every supported provider family or is explicitly capability-gated.
- Provider failures degrade cleanly.
- Cost is visible and enforceable before a batch or long-run exceeds budget.

13 11. Phase 6 - Candidate truth, memory, prompts and evidence

13.1 11.1 Candidate Truth System

Introduce first-class `CandidateFact` records:

```
fact_id
category
value
source
evidence
```


confidence
valid_from
valid_until

Sources can include:

- user-entered profile.
- imported resume.
- portfolio/project artifact.
- verified external profile.
- prior application evidence.

LLM output may propose a fact, but it must not silently mutate authoritative truth.

13.2 11.2 Grounded application answering

Classify questions as:

FACTUAL
PREFERENCE
ELIGIBILITY
LEGAL
SENSITIVE
FREE_TEXT
UNKNOWN

Workflow:

Question
-> classifier
-> known candidate fact?
 yes -> grounded answer
 no -> policy
 -> LLM proposal
 -> grounding verifier
 -> approval if needed

Never invent material candidate facts.

13.3 11.3 Layered memory

Separate:

Hot	current task/application
Warm	active search preferences
Semantic	stable candidate facts
Episodic	what happened in application X
Procedural	how a site workflow works

Historical past applications/outcomes

External company/market knowledge

Every durable memory record needs provenance, confidence and versioning.

13.4 11.4 Memory retention and privacy

Define retention classes and purge behavior. Do not retain raw screenshots, resumes, or employer data forever by default.

Support:

- export.
- purge by entity.
- purge by date.
- full reset.
- backup encryption.
- redacted logs.

13.5 11.5 Prompt registry

Store prompts in versioned files/modules:

```
prompts/  
  application/  
  matching/  
  documents/  
  qa/  
  interview/  
  outreach/
```

Every model call should record:

- prompt id/version.
- model/provider.
- relevant schema version.
- profile version.
- tool/skill versions.
- policy version.

13.6 11.6 Evidence system

For jobs capture:

- source URL.
- timestamp.
- raw posting or reproducible snapshot.
- normalized representation.
- parser version.

For applications capture:

- original job description.
- match evaluation.
- tailored artifacts.
- submitted values.
- evidence snapshots.
- confirmation.
- verification.
- outcome.

Evidence must be navigable from GUI and trace views.

13.7 11.7 Resume/cover pipeline

Required sequence:

Job Description

- > parse
- > deterministic fit
- > candidate evidence
- > tailoring plan
- > draft
- > independent review
- > revision
- > PDF build
- > text-layer verification
- > ATS verification
- > visual verification
- > final artifact

Reviewers must catch:

- unsupported claims.
- keyword stuffing.
- contradictions.
- formatting failures.
- broken PDF text extraction.
- ATS parseability failures.

14 12. Phase 7 - Discovery, matching, fraud checks and career intelligence

14.1 12.1 Multi-stage job matching

Use increasingly expensive stages:

Stage 1: deterministic eligibility filter
Stage 2: lexical/embedding score
Stage 3: structured LLM evaluation
Stage 4: deep company/job research for shortlist
Stage 5: recommendation

Dimensions:

- skills.
- seniority.
- location.
- visa/work authorization.
- compensation.
- employment type.
- industry.
- company preferences.
- remote/hybrid constraints.
- career trajectory fit.

Always provide a match explanation.

14.2 12.2 Job quality and fraud detection

Score signals such as:

- inconsistent company identity.
- suspicious domain.
- payment/fee requests.
- credential requests unrelated to normal hiring.
- implausible compensation.
- recycled job text.
- suspicious recruiter contact patterns.
- missing company information.
- mismatched application domains.

Never claim certainty when the signal is probabilistic.

14.3 12.3 Company normalization

Create canonical entities for:

- company.
- domain.
- aliases.
- subsidiaries.
- locations.
- recruiters.
- jobs.

Use identity confidence and provenance.

14.4 12.4 Proactive job discovery

Support configurable background scanning that:

- polls allowed sources.
- deduplicates.
- ranks jobs.
- alerts on high matches.
- optionally pre-generates preparation artifacts, but does not submit without policy.

Provide a weekly opportunity digest.

14.5 12.5 Application outcome learning

Track:

job
application
source
match score
resume variant
cover letter variant
timing
networking/referral path
follow-up
response
interview
offer/rejection

Use this to estimate:

- which job sources convert.
- which role families convert.
- which resume variants convert.
- which skills correlate with response.
- which timing patterns correlate with response.

Avoid naive causality claims. Use confidence intervals, sample-size warnings, and observational wording.

14.6 12.6 Career intelligence graph

Long term, model:

Candidate
-> Skill Graph

- > Market Graph
- > Job Graph
- > Company Graph
- > Outcome Graph

Enable questions such as:

- what roles are best aligned now?
- why am I not getting interviews?
- which skill is the highest-value next investment?
- which companies respond most often?
- which role families have the best expected return?

14.7 12.7 Networking and referral tracking

Implement:

- contacts.
- companies.
- recruiters.
- relationships.
- outreach activities.
- referral sources.
- follow-up schedules.

Recommend warm introductions without auto-contacting people by default.

15 13. Phase 8 - GUI, TUI, control plane and reports

15.1 13.1 GUI is the control plane

The Tauri app should become the operational cockpit, not just a demo shell.

15.1.1 Home

Show:

- active work.
- pending approvals.
- failures/incidents.
- applications today.
- cost today.
- top matches.
- system health.

15.1.2 Task inspector

Show:

- status.
- owner/worker.
- attempts.
- dependencies.
- phase.
- evidence.
- logs.
- cost.
- verification.

15.1.3 Application detail

Show:

- job.
- match score.
- match reasons.
- resume.
- cover letter.
- answers.
- submission state.
- screenshots.
- confirmation.
- verification evidence.
- timeline.

15.1.4 Approval inbox

Show:

WHAT

WHY

RISK

EVIDENCE

POLICY

BUDGET

Actions:

- approve.
- edit.
- reject.
- defer.
- cancel.

15.1.5 Incident console

Show:

- what happened.
- timeline.
- affected jobs/applications.
- probable root cause.
- current mitigation.
- recommended human action.
- suggested improvement candidate.

15.2 13.2 GUI feature backlog

Integrate:

- Kanban tracker.
- application funnel analytics.
- persistent screening-answer bank.
- resume variants.
- live ATS score.
- job browser.
- job detail.
- job research view.
- profile editor.
- interview coach.
- networking/contacts view.
- cost dashboard.
- audit log.
- trace viewer.
- selector health/adaptor diagnostics.
- backup/export settings.
- privacy/telemetry controls.
- incident recovery UI.

Market-facing job tracker systems currently emphasize centralized job/application tracking, documents, contacts, activities and metrics; these are table-stakes usability features, but JoBot should integrate them into its durable execution/control-plane model rather than clone the UX.

15.3 13.3 TUI

Implement a `textual`-based terminal dashboard with:

- task queue.
- application queue.
- current worker state.
- live scrape progress.

- saga status.
- LLM cost tracker.
- approval prompts.
- errors and incidents.

15.4 13.4 HTML reports

Enhance tracker reports with:

- funnel visualization.
- weekly trends.
- response rates.
- resume variant metrics.
- source performance.
- cost per application.
- time-to-application.

Reports must be generated from durable data and cite their time window and data completeness.

16 14. Phase 9 - CLI and repository-wide refactor

16.1 14.1 CLI decomposition

Split the large CLI module into:

```
cli/
  apply.py
  scrape.py
  resume.py
  interview.py
  tracker.py
  profile.py
  config.py
  admin.py
  doctor.py
  helpers.py
  main.py
```

Preserve command signatures unless a breaking version explicitly permits change.

Target: `main.py` becomes a thin registration layer.

16.2 14.2 Core service boundaries

Move domain logic out of CLI adapters into reusable services. The same services must be callable from:

- CLI.
- GUI sidecar.
- TUI.
- API server.
- MCP.
- scheduled tasks.
- tests.

16.3 14.3 Adapter isolation

Each adapter must depend on typed contracts, not UI behavior or global mutable state.

16.4 14.4 Database refactor

Split storage responsibilities conceptually:

```
connection / transaction
schema / migrations
repositories
projections
backup / restore
queries / analytics
```

Do not scatter raw SQL through business logic.

16.5 14.5 Error model

Replace generic exceptions and silent `except` `Exception: pass` patterns with typed error categories:

```
ConfigurationError
ValidationError
PolicyDenied
AuthenticationRequired
RateLimited
TransientNetworkError
AdapterProtocolError
BrowserDriftError
VerificationFailed
ExternalEffectUnknown
QuotaExceeded
DependencyUnavailable
```

MigrationError
SecurityViolation

Every error maps to:

- user-facing message.
- retry policy.
- observability severity.
- quarantine/incident behavior.

16.6 14.6 Structural refactor strategy

Use two tracks:

16.6.1 Track A - low-regression refactor

- decompose CLI.
- add contracts.
- isolate dependencies.
- add tests.
- wire currently-unwired subsystems.
- eliminate dead code.

16.6.2 Track B - deep restructure

- move packages into durable architecture layers.
- introduce event bus.
- plugin-ify adapters.
- add MCP.
- async-convert hot paths.
- add multi-worker coordination only after single-agent baseline is stable.

Never perform Track B while Track A is unstable.

17 15. Phase 10 - MCP, API server, plugins and integrations

17.1 15.1 MCP as an interoperability boundary

MCP must stay outside JoBot's core domain model. Treat it as a thin adapter over the same approved service methods used by the GUI/CLI.

Expose tools such as:

- search_jobs.
- get_job.
- rank_jobs.

- `get_candidate_profile`.
- `generate_resume`.
- `get_application`.
- `prepare_application`.
- `request_application_approval`.
- `get_application_evidence`.
- `get_search_analytics`.
- `doctor`.

Also consider resources and prompts where useful, but do not expose raw private storage by default.

The MCP ecosystem has evolved rapidly. As of the July 28, 2026 specification, the protocol includes a stateless core option, multi-round-trip requests, header-based routing, cacheable list results, and authorization hardening. The implementation must pin/track a concrete supported MCP revision and treat protocol upgrades as compatibility work, not as an incidental dependency bump.

17.2 15.2 API server mode

Add optional `jobot serve` with a minimal FastAPI surface for:

- `health`.
- `job discovery`.
- `applications`.
- `approvals`.
- `profiles`.
- `tracker`.
- `traces`.
- `exports`.

Security requirements:

- localhost-only by default.
- API-key or equivalent auth for exposed mode.
- rate limiting.
- explicit CORS.
- no secret-bearing endpoints without authorization.

17.3 15.3 Plugin architecture

Every plugin requires a manifest declaring:

- `name/version`.
- `source/hash`.
- `capabilities`.
- `permissions`.
- `dependencies`.
- `network targets`.

- browser targets.
- secrets required.
- isolation mode.

Default permissions are deny.

Install flow:

```
manifest
-> hash/signature verification
-> dependency audit
-> permission review
-> sandbox selection
-> install
-> health check
-> register
```

17.4 15.4 Browser extension

Add only after stable local API/control-plane contracts exist.

Capabilities:

- save current job.
- send page URL and structured job data to JoBot.
- request autofill.
- show prepared answers.
- require approval before external submit.

Do not make the extension a bypass for platform controls.

18 16. Phase 11 - Production packaging, release engineering and operations

18.1 16.1 One version authority

Choose `pyproject.toml` or a dedicated release file as the sole semantic version authority. Generate/update:

- Python package version.
- CLI version.
- GUI package version.
- Tauri version.
- Docker tags.
- release notes metadata.

Add a CI job that fails on version drift.

18.2 16.2 Python distribution

Produce:

- wheel.
- sdist.
- optional feature extras.

Use PyPI Trusted Publishing with GitHub Actions OIDC rather than long-lived upload tokens. PyPI documents trusted publishing as a short-lived OIDC exchange that removes the need to store a long-lived API token in CI.

18.3 16.3 Docker

Provide:

- development image.
- production image.
- CI image.
- mock/test image where useful.

Hardening:

- non-root where feasible.
- minimal base image.
- resource limits.
- persistent volume documentation.
- health endpoint.
- no default credential persistence.
- explicit browser/runtime installation.

Build and publish multi-architecture images where practical.

18.4 16.4 Desktop

Target:

- Windows.
- macOS.
- Linux.

Produce reproducible CI artifacts and document signing/notarization state honestly.

Use Tauri updater only with signed artifacts and verified public keys.

18.5 16.5 Release channels

nightly
alpha
beta

rc
stable

Semantic versioning rules must be documented.

18.6 16.6 Release pipeline

PR

- > lint
- > format
- > type check
- > unit tests
- > integration tests
- > GUI tests
- > security tests
- > eval suite
- > package build
- > desktop build
- > Docker build
- > SBOM
- > provenance
- > RC artifact smoke test
- > release
- > publish
- > post-release verification

18.7 16.7 Release gates

18.7.1 Functional

- unit tests.
- integration tests.
- critical evals.
- adapter contract tests.
- CLI smoke.
- GUI smoke.

18.7.2 Reliability

- crash/recovery.
- worker lease recovery.
- duplicate-submission.
- provider fallback.
- approval/resume.
- browser reconnect.
- DB corruption detection.

18.7.3 Security

- CodeQL.
- dependency audit.
- secret scan.
- plugin permission tests.
- prompt-injection tests.
- credential leakage tests.
- URL parsing tests.
- Tauri capability tests.

18.7.4 Packaging

- install from wheel.
- install from sdist.
- Docker smoke.
- desktop launch.
- jobot doctor.
- migration from prior release.
- upgrade/downgrade behavior according to policy.

18.8 16.8 jobot doctor

The doctor command is release-critical and should verify:

```
runtime
python/node
sqlite/filesystem
security/keyring/vault
browser
chromium/profile availability
document/PDF engines
LLM provider configuration
adapter registry
migrations
event store
queues/workers
package metadata
```

Support `--json` for support automation.

18.9 16.9 Backup and recovery

Provide:

```
jobot backup
jobot restore
jobot verify-backup
```



```
jobot db status
jobot db migrate
jobot db verify
```

Backup should include:

- database state.
- encrypted profile data.
- required durable artifacts.
- memory.
- configuration metadata.

Do not include transient caches unless explicitly requested.

19 17. Phase 12 - Observability, privacy and governance

19.1 17.1 OpenTelemetry

Adopt OpenTelemetry as the standard telemetry abstraction for traces, metrics and logs, while keeping local-first storage primary and external export optional. OpenTelemetry is explicitly vendor-neutral and supports traces, metrics and logs.

Trace hierarchy:

Goal

```
-> Task
    -> Model call
    -> Tool call
    -> Browser action
    -> Policy evaluation
    -> Approval
    -> Verification
    -> Artifact
```

Every trace should carry:

- trace id.
- run id.
- goal id.
- task id.
- application id.
- worker id.
- provider/model.
- prompt version.
- policy version.

- adapter version.

19.2 17.2 Structured logging

Use structured JSONL in production and readable logs in development. Include correlation ids. Support log rotation and retention limits.

19.3 17.3 Audit log

Maintain an append-only consequential-action log with:

- timestamp.
- actor.
- action.
- target.
- outcome.
- policy result.
- effect id.
- evidence.

Add tamper-evident hashing for the log chain.

19.4 17.4 Telemetry privacy

Telemetry must be opt-in.

Redact:

- candidate identity.
- API keys.
- company/job URLs when they can identify sensitive information.
- evidence paths.
- application content.
- secrets.

Provide:

- GUI consent.
- `jobot config set telemetry.enabled true|false`.
- environment kill switch.
- `jobot purge`.
- exact privacy documentation.

19.5 17.5 Privacy documentation

`docs/privacy.md` must say:

- what is collected.
- what is not collected.
- when collection occurs.

- where it is sent.
- retention.
- how to disable.
- how to delete local data.

Automate schema tests so docs and telemetry payloads cannot drift silently.

20 18. Phase 13 - Evaluation platform and agent reliability science

Treat evals as a first-class product subsystem and release gate.

20.1 18.1 Capability evals

Cover:

- job discovery.
- parsing.
- matching.
- resume tailoring.
- cover letters.
- QA.
- form filling.
- application submission.
- verification.
- tracker updates.

20.2 18.2 Reliability evals

Track:

- pass@1.
- repeat-run pass.
- pass@N.
- median duration.
- p95 duration.
- retry count.
- intervention rate.
- silent failure rate.
- unknown-state rate.
- evidence completeness.

20.3 18.3 Security evals

Include:

- prompt injection from job descriptions.
- malicious HTML.
- malicious URLs.
- fake jobs.
- credential exfiltration requests.
- secret leakage in prompts/logs/events.
- destructive tool requests.
- malicious plugins.
- compromised adapter responses.

20.4 18.4 Long-horizon evals

Example:

Find 20 jobs

```
-> filter
-> rank
-> shortlist 3
-> tailor 3 resumes
-> generate cover letters
-> answer questions
-> request approvals
-> submit authorized applications
-> verify
-> update tracker
-> update memory
-> report outcome
```

Inject process kills between phases.

20.5 18.5 Failure injection

Inject:

- browser crash.
- network loss.
- 429.
- 500.
- malformed JSON.
- provider timeout.
- provider outage.
- SQLite lock.
- process kill.
- worker loss.
- stale browser session.
- selector drift.
- CAPTCHA detection.

- application ambiguity.

20.6 18.6 Regression policy

Every release compares against the last accepted baseline. A change that improves feature quality but increases unsafe behavior, cost, or intervention rate must not be treated as a pure improvement.

21 19. Phase 14 - Self-improvement, skill extraction and bounded autonomy

21.1 19.1 Capability acquisition loop

Use:

```
solve once
-> repeatable trajectory
-> skill
-> workflow
-> harness
-> evals
-> automation
-> monitoring
-> trusted autonomy
```

21.2 19.2 Skill extraction

Successful trajectories may become skill candidates.

Example:

```
workday_application_v1
```

could encode:

- navigation.
- selectors.
- field classifications.
- question types.
- failure modes.
- recovery strategies.
- verification behavior.

Each skill needs a regression corpus.

21.3 19.3 Bounded self-improvement

Use:

Production failure

- > classify gap
- > propose improvement
- > branch/sandbox
- > run evals
- > compare baseline
- > human/policy gate
- > promote or discard

The agent must never directly rewrite production code or policy based solely on its own judgment.

21.4 19.4 Trust promotion/demotion

A supervised integration may be promoted only after measurable evidence of stable successful behavior.

Suggested model:

- promotion after a threshold of successful verified runs.
- demotion after consecutive failures, unusual verification rates, policy exceptions or user overrides.
- trust changes recorded as audit events.

Thresholds should be configurable and learned from evidence, not hard-coded as product guarantees.

22 20. Phase 15 - Data model and migration overhaul

22.1 20.1 Versioned migrations

Replace ad-hoc migrations with:

```
schema_migrations
  version
  applied_at
  checksum
```

Example sequence:

```
001_initial
002_events
003_task_leases
```

004_approvals
005_budgets
006_effects
007_memory_provenance
008_candidate_facts
009_job_entities
010_outcome_timestamps

22.2 20.2 Repository layer

Implement repositories for:

- goals.
- tasks.
- events.
- leases.
- approvals.
- applications.
- effects.
- jobs.
- profiles.
- memory.
- traces.
- budgets.
- incidents.

22.3 20.3 Indexing strategy

Measure before indexing, but likely candidates include:

- task status + lease expiry.
- application idempotency key.
- application profile + status.
- event aggregate + created_at.
- job normalized identity.
- approval state + expiry.
- effect idempotency key.
- LLM call timestamp/provider.

22.4 20.4 SQLite durability settings

Use WAL intentionally. Configure:

- busy timeout.
- transaction boundaries.
- checkpoint strategy.
- backup integration.

- bounded WAL growth.

Do not introduce a remote database requirement merely to solve problems SQLite can handle for the supported local-first use case.

23 21. Phase 16 - Performance and resource-efficiency program

This section is mandatory. The goal is not merely “faster” code; it is lower total resource use per successful verified outcome.

23.1 21.1 Baseline before optimization

Measure:

- cold start latency.
- warm start latency.
- peak RSS.
- idle RSS.
- CPU usage at idle and active.
- SQLite query latency.
- DB file size growth.
- WAL growth.
- browser startup cost.
- browser session reuse rate.
- LLM tokens per workflow.
- cache hit rate.
- model cost per successful verified application.
- artifact storage per application.
- GUI memory footprint.
- sidecar message throughput.

Record representative workloads for:

1. doctor.
2. discover 100 jobs.
3. rank 100 jobs.
4. prepare 1 application.
5. prepare 10 applications.
6. run a browser-assisted application.
7. long-running scheduler.
8. GUI idle.
9. GUI active.

23.2 21.2 Python efficiency

- lazy-import heavy optional dependencies.
- separate optional provider/scrapper extras.
- avoid repeated model construction.
- reuse HTTP sessions.
- batch SQLite writes.
- avoid N+1 queries.
- avoid serial JSON encode/decode cycles.
- prefer iterators for large result sets.
- cache immutable normalized data.
- use bounded concurrency.

23.3 21.3 Browser efficiency

- reuse sessions where safe.
- reuse browser context rather than launching a browser per action.
- stop recording unnecessary full-page screenshots.
- capture targeted evidence.
- compress images.
- close inactive contexts.
- enforce timeouts.
- use deterministic waits instead of arbitrary sleeps.

23.4 21.4 LLM efficiency

Use the cost ladder:

deterministic

- > cheap semantic
- > small structured model
- > expensive reasoning model

Additional techniques:

- prompt compression.
- result caching.
- exact-request deduplication.
- embedding caching.
- artifact references instead of repeated content.
- structured output to reduce retries.
- batch independent calls.
- provider routing by task requirements.
- local model fallback for low-risk workloads.

23.5 21.5 Storage efficiency

- deduplicate job postings.

- deduplicate immutable artifacts by content hash.
- compress evidence.
- separate hot metadata from bulky artifacts.
- expire stale browser logs.
- compact telemetry.
- avoid storing both full and derived representations unless justified.

23.6 21.6 GUI efficiency

- virtualize long lists.
- avoid unnecessary global state updates.
- debounce queries.
- paginate large timelines.
- stream only active task updates.
- cache static metadata.
- avoid loading all evidence images until requested.

23.7 21.7 Sidecar efficiency

- bounded message buffers.
- backpressure.
- correlation ids without excessive duplicated payload.
- restartable process supervision.
- no unbounded stdout capture.

23.8 21.8 Performance gates

Establish budgets only after baseline measurement. A release should require no material regression in:

- startup.
- peak memory.
- CPU saturation.
- database growth.
- token usage per successful run.
- application verification latency.

Where improvements are targeted, use before/after benchmarks and retain the report as a release artifact.

24 22. Phase 17 - Repository cleanup and unnecessary-file removal

This section must be executed as a dedicated work package rather than informal deletion during refactors.

24.1 22.1 Canonical top-level structure

Converge toward:

```
AGENTS.md
README.md
LICENSE
SECURITY.md
CONTRIBUTING.md
CODE_OF_CONDUCT.md
CHANGELOG.md
pyproject.toml
package.json
package-lock.json
Dockerfile
docker-compose.yml
src/
tests/
gui/
docs/
scripts/
queues/
state/
.github/
```

Generated artifacts should not accumulate at root.

24.2 22.2 Documentation consolidation

Create:

```
docs/
  README.md
  architecture.md
  contracts.md
  security.md
  privacy.md
  release-policy.md
  release-runbook.md
  cli-reference.md
  api-reference.md
  gui-guide.md
  adapters.md
  memory.md
  evaluation.md
  troubleshooting.md
  faq.md
  migration.md
```

```
development.md
decisions/
history/
implementation/
research/
```

Move superseded plans into `docs/history/` or `docs/history/plans/` after content hashes and provenance are recorded.

24.3 22.3 Duplicate detection

Find duplicates by:

- exact hash.
- normalized textual hash.
- semantic similarity where necessary.

Do not delete two documents merely because titles are similar. Preserve unique content.

24.4 22.4 Generated-file cleanup

Remove tracked/generated files such as:

- stale build outputs.
- coverage fragments.
- temporary PDFs.
- obsolete GUI bundles.
- test caches.
- stale screenshots.
- obsolete binary release assets.

Do not remove user data, runtime state, or evidence without a retention/migration policy.

24.5 22.5 Secret-like file audit

Ensure:

- `.env` is never tracked.
- certificates/keys are ignored.
- dumps/backups are controlled.
- generated evidence containing PII has a retention policy.

If a tracked secret is discovered:

1. remove it.
2. rotate it.
3. scan history.
4. document the incident.

24.6 22.6 Plan/document authority

There must be exactly one active implementation plan. Historical plans are immutable references, not active instructions.

24.7 Exit criteria

- `git status` clean except intentional changes.
 - no unexplained duplicate plans.
 - no tracked secrets.
 - no stale authoritative README claims.
 - every large historical file has an intentional location and reason.
-

25 23. Phase 18 - Documentation generation program

Documentation is part of the product, not post-processing.

25.1 23.1 Required documentation set

Generate and maintain:

25.1.1 User documentation

- Quickstart.
- Installation per OS.
- `jobot doctor` guide.
- profile onboarding.
- resume import.
- job discovery.
- matching explanation.
- application preparation.
- approval workflow.
- application verification.
- tracker.
- interview coach.
- networking.
- export/purge.
- troubleshooting.

25.1.2 Developer documentation

- architecture.
- module map.

- service boundaries.
- contracts.
- state machine.
- event schema.
- effect ledger.
- database schema.
- migration policy.
- plugin ABI.
- adapter development.
- selector registry.
- browser reliability.
- prompt registry.
- model routing.
- eval framework.
- telemetry.
- security threat model.

25.1.3 Operations documentation

- local runbook.
- release runbook.
- incident response.
- backup/restore.
- migration/upgrade.
- adapter health.
- dependency update policy.
- security disclosure.
- telemetry/privacy.
- signing/publishing.

25.2 23.2 Generated references

Generate where practical from source-of-truth definitions:

- CLI reference from command metadata/tests.
- API schema from FastAPI/OpenAPI.
- MCP tool catalogue from server registration.
- adapter capability matrix from registry metadata.
- config reference from typed settings.
- event catalogue from typed event definitions.
- migration list from migration files.

25.3 23.3 Documentation verification

CI should:

- build docs.

- validate links.
- verify code snippets where practical.
- verify command examples against the current CLI.
- verify version numbers.
- verify screenshots are current.
- check that release claims match actual supported adapters.

26 24. Feature integration matrix - complete unified backlog

This matrix merges the source plans without making every feature P0.

ID	Capability	Priority	Dependency	Notes
C-01	Durable task queue	P0	DB/migrations	Foundational
C-02	Atomic leases	P0	C-01	Crash recovery
C-03	Heartbeats	P0	C-02	Worker safety
C-04	Event ledger	P0	C-01	Audit/replay
C-05	Event bus	P1	C-04	Control-plane integrations
C-06	Effect ledger	P0	C-04	Duplicate prevention
C-07	Idempotency audit	P0	C-06	All side effects
C-08	Unknown state model	P0	C-06	Reconciliation
C-09	Durable approvals	P0	C-04	CLI/GUI/MCP
C-10	Risk/trust engine	P0	C-09	Scoped autonomy
C-11	Browser session manager	P0	C-09	Application flows
C-12	Selector registry/healing	P0	C-11	Drift handling
C-13	Browser evidence	P0	C-11	Verification

ID	Capability	Priority	Dependency	Notes
C-14	Adapter boundary schemas	P0	C-04	Contract safety
C-15	ATS family abstraction	P1	C-14	Workable etc.
C-16	LinkedIn assisted flow	P1	C-11	ToS/authorization gated
C-17	Authorized direct API apply	P1	C-14	Credentials required where applicable
C-18	LLM streaming	P0	provider interfaces	Current stub
C-19	Capability- aware routing	P0	durable cost ledger	Economics
C-20	Prompt versioning	P0	LLM calls	Eval correlation
C-21	Candidate facts	P0	profile/memory	Truth system
C-22	Layered memory	P0	C-21	Provenance
C-23	Independent resume reviewer	P0	docs pipeline	Quality gate
C-24	PDF text verification	P0	docs pipeline	Release gate
C-25	ATS/visual resume verification	P0	C-24	Evidence
C-26	Multi-stage job match	P0	discovery	Cost control
C-27	Job fraud signals	P1	job normal- ization	Safety
C-28	Company normaliza- tion	P1	discovery	Analytics
C-29	Conversational assistant	P1	services/API	Read-heavy initially
C-30	Resume A/B testing	P1	outcomes	Data-driven
C-31	LinkedIn profile analyzer	P1	authorized data	Compliance- aware

ID	Capability	Priority	Dependency	Notes
C-32	GUI dashboard	P0	control plane	Release UX
C-33	GUI Kanban	P0	applications	Table-stakes tracker
C-34	Approval inbox	P0	approvals	Safety UX
C-35	Evidence viewer	P0	evidence	Trust UX
C-36	Trace viewer	P1	telemetry	Debugging
C-37	Cost dashboard	P1	LLM ledger	Economics
C-38	Incident console	P1	events/telemetry	Recovery
C-39	TUI	P1	services	Power users
C-40	HTML reports	P1	tracker	Export
C-41	OpenTelemetry	P0	events/traces	Observability
C-42	Opt-in telemetry	P1	privacy	Optional
C-43	jobot doctor --json	P0	services	Supportability
C-44	Version authority	P0	packaging	Release blocker
C-45	Versioned DB migrations	P0	storage	Upgrade safety
C-46	PyPI trusted publishing	P0	CI	Supply chain
C-47	Docker publish	P0	CI	Distribution
C-48	Desktop CI	P0	Tauri hardening	Distribution
C-49	Updater/signing	P1	desktop CI	OS-specific
C-50	Browser extension	P2	local API	Later
C-51	MCP server	P1	service contracts	Interop
C-52	API server	P1	service contracts	Headless
C-53	Plugin permissions	P1	plugin registry	Security

ID	Capability	Priority	Dependency	Notes
C-54	Sandbox execution	P1	execution layer	Untrusted code
C-55	Proactive discovery	P1	scheduler	Career OS
C-56	Follow-up automation	P1	contacts	ToS/email gated
C-57	Networking graph	P1	contacts	Referral tracking
C-58	Market intelligence	P2	external data	Long-term
C-59	Outcome learning	P1	analytics	Statistical care
C-60	Skill extraction	P2	trajectories/evals	Moor
C-61	Bounded self-improvement	P2	evals/sandbox	Never unrestricted
C-62	Multi-machine workers	P2	durable core	Later
C-63	Multi-language resumes	P2	docs/profile	Region-specific
C-64	Resume import	P0	profile	Onboarding
C-65	Screening answer bank	P0	candidate truth	Reuse
C-66	Data export/import	P0	storage	Portability

27 25. Competitive feature synthesis

The product should absorb underlying capabilities from competing systems, not copy proprietary implementations.

Current commercial products emphasize centralized tracking, job bookmarking, personalized matching, tailored resumes, ATS optimization, autofill, contacts, metrics and career insights. Jobright currently markets job matching, autofill, tailored resumes and insider connections, while Teal emphasizes tailored resumes, job tracking, ATS checks, profile import and job matching. Simplify emphasizes a job tracker integrated with applications and an extension-driven

autofill experience. These confirm that workflow continuity is table stakes for a serious job-search product.

JoBot’s differentiation should remain:

1. durable verified execution rather than fragile macros.
 2. local-first privacy rather than mandatory hosted processing.
 3. candidate-truth grounding rather than generic generation.
 4. evidence-backed submission verification.
 5. human-governed autonomy rather than opaque “set and forget” execution.
 6. cost-aware multi-model routing.
 7. auditable trajectories and release-gated evaluations.
 8. extensible career-OS architecture rather than a single board integration.
-

28 26. P0 backlog - release-blocking work

The following is the minimum release-blocking layer.

28.1 Correctness

- ☐ Persistent task queue.
- ☐ Atomic leases.
- ☐ Worker heartbeats.
- ☐ Persistent task attempts.
- ☐ Event ledger.
- ☐ Durable checkpoints.
- ☐ Durable approvals.
- ☐ Effect ledger.
- ☐ Unknown states.
- ☐ Submission reconciliation.
- ☐ Idempotency audit.
- ☐ State-machine transition validation.

28.2 Security

- ☐ URL inference hardening.
- ☐ Vault hardening.
- ☐ Tauri CSP/capability hardening.
- ☐ Prompt-injection boundary.
- ☐ Credential isolation.
- ☐ Plugin permissions.
- ☐ Secret redaction.
- ☐ Security eval suite.

28.3 Verification

- ☐ Independent verifier.
- ☐ PDF text-layer verification.
- ☐ PDF visual verification.
- ☐ Evidence/trace correlation.
- ☐ Failure injection.
- ☐ Crash/restart recovery.

28.4 Release

- ☐ Single version authority.
 - ☐ Versioned migrations.
 - ☐ Clean install tests.
 - ☐ Wheel/sdist tests.
 - ☐ Docker smoke.
 - ☐ Desktop build.
 - ☐ PyPI trusted publishing.
 - ☐ RC pipeline.
 - ☐ Upgrade test.
 - ☐ Rollback/recovery procedure.
-

29 27. P1 backlog - v1.x product expansion

- ☐ JobSpy and multi-board aggregation improvements.
- ☐ ATS family expansion.
- ☐ Company entity graph.
- ☐ Better matching.
- ☐ Career scoring.
- ☐ Kanban/funnel analytics.
- ☐ Follow-ups.
- ☐ Email synchronization.
- ☐ Networking.
- ☐ Salary intelligence.
- ☐ Fraud detection.
- ☐ MCP.
- ☐ API server.
- ☐ OpenTelemetry export.
- ☐ Optional external tracing.
- ☐ Sandbox execution.
- ☐ Browser skill registry.
- ☐ Application analytics.
- ☐ Proactive discovery.
- ☐ Resume A/B testing.

- ☐ LinkedIn profile analysis where data access is authorized.
-

30 28. P2 strategic moat

- ☐ Outcome-based job matching.
 - ☐ Career opportunity graph.
 - ☐ Automatic skill extraction.
 - ☐ Trajectory mining.
 - ☐ Automated eval generation.
 - ☐ Bounded self-improvement.
 - ☐ Adaptive routing.
 - ☐ Personalized career strategy.
 - ☐ Market intelligence.
 - ☐ Skill-gap planning.
 - ☐ Multi-machine workers.
 - ☐ Remote sandbox.
 - ☐ Plugin ecosystem.
 - ☐ Multi-language resumes.
 - ☐ Browser extension.
-

31 29. First production milestone - Durable Verified Application

Nothing should be called v1.0-ready until this sequence works end to end on a supported workflow.

1. Resolve job
2. Persist job
3. Create goal/task
4. Evaluate policy
5. Evaluate fit
6. Build candidate evidence set
7. Generate tailored resume
8. Generate cover letter
9. Independent reviewer validates materials
10. Compile PDF
11. Verify PDF text layer
12. Verify visual PDF
13. Request human approval
14. Persist approval waitpoint
15. Resume after approval

16. Open/enter authorized submission path
17. Fill application
18. Reserve external effect idempotently
19. Submit once
20. Capture evidence
21. Verify confirmation or remote state
22. Mark VERIFIED or UNKNOWN
23. Persist outcome
24. Update memory
25. Emit trace
26. Update metrics
27. Generate bounded improvement candidate

Then deliberately kill the process:

- before preparation.
- after preparation.
- before approval.
- while waiting for approval.
- immediately before submission.
- immediately after submission.
- before verification.
- after verification.

The agent must demonstrate correct recovery in each case without duplicate external effects.

32 30. Release-readiness definition of done

JoBot is release-ready only when all of the following are true.

32.1 Product

- End-to-end supported application workflow works.
- Average user can onboard without editing source code.
- Profile import works.
- Matching is explainable.
- Documents are verified.
- Approval flow is understandable.
- Application tracking is usable.
- Recovery is observable.

32.2 Architecture

- Durable task graph.

- Durable approvals.
- Durable events.
- Effect ledger.
- Versioned migrations.
- Typed adapter contracts.
- Explicit state machine.
- Unknown states.

32.3 Security

- Secret scan clean.
- Dependency audits clean or accepted with documented residual risk.
- URL parsing secure.
- Tauri capabilities least privilege.
- Prompt injection boundary evaluated.
- Plugin permissions enforced.
- Telemetry redacted.

32.4 Reliability

- Failure injection passes.
- Crash recovery passes.
- Duplicate submission tests pass.
- Browser/session recovery passes.
- Provider fallback passes.
- Scheduler edge cases pass.
- Backup/restore passes.

32.5 Observability

- Structured logs.
- traces.
- metrics.
- audit log.
- incident handling.
- cost tracking.
- evidence viewer.

32.6 Packaging

- wheel.
- sdist.
- Docker.
- desktop installers.
- signed/provenance metadata where available.
- trusted publishing.

- updater verified.

32.7 Documentation

- README current.
- install guide.
- CLI reference.
- GUI guide.
- security.
- privacy.
- release policy.
- contributor docs.
- adapter docs.
- troubleshooting.

32.8 Governance

- SECURITY policy.
- code of conduct.
- contribution guide.
- issue templates.
- release runbook.
- clear license metadata.

32.9 Evidence

A release dossier must contain:

```
baseline report
full CI result
security scan result
eval report
failure-injection report
performance report
package verification report
desktop smoke report
backup/restore report
version consistency report
documentation build report
```

33 31. Agent execution protocol

This section is the operational harness for implementing the plan.

33.1 31.1 Before every task

Read:

- relevant source code.
- related tests.
- current contract.
- related queue/state files.
- dependency constraints.

Write down:

- intended change.
- why.
- affected modules.
- risks.
- verification command.
- rollback/recovery approach.

33.2 31.2 During implementation

Prefer:

```
one change
-> tests
-> inspect output
-> update state
-> next change
```

Avoid large speculative rewrites.

33.3 31.3 After implementation

Every task must update:

- code.
- tests/evals.
- documentation if behavior changed.
- worklog.
- relevant queue.
- decisions if an architecture decision was made.

33.4 31.4 Worktree isolation

For parallel implementation:

- give each worker an explicit task boundary.
- avoid overlapping file ownership where possible.
- use separate worktrees for coding tasks that can conflict.
- merge only after independent tests pass.

Do not create a swarm merely for the sake of parallelism.

33.5 31.5 Worker timeouts and heartbeats

Use bounded execution.

Default behavior:

- regular heartbeat.
- checkpoint before expensive operations.
- bounded retry.
- explicit cancellation.
- quarantine if recovery cannot be proven.

33.6 31.6 Approval boundary

The agent itself must not silently promote a task to a higher autonomy level because implementation appears reliable. Trust changes require explicit policy and evidence.

33.7 31.7 No fake completion

Do not report:

- “verified” without evidence.
 - “production-ready” without release gates.
 - “supports X” without a tested adapter/flow.
 - “secure” without scan/eval evidence.
 - “fast” without benchmarks.
 - “private” without data-flow inspection.
-

34 32. Suggested workstream decomposition

Use these workstreams only where parallelism is genuinely independent.

34.1 WS-A - Durable core

Tasks:

- migrations.
- events.
- task leases.
- checkpoints.
- effect ledger.
- approval entities.

34.2 WS-B - Security and release

Tasks:

- dependency upgrades.
- CodeQL fixes.
- Tauri security.
- vault.
- CI.
- governance.

34.3 WS-C - Application reliability

Tasks:

- application state machine.
- browser sessions.
- selector registry.
- evidence.
- reconciliation.

34.4 WS-D - AI and documents

Tasks:

- streaming.
- routing.
- prompt registry.
- candidate facts.
- memory.
- document verification.

34.5 WS-E - Product surface

Tasks:

- GUI.
- TUI.
- tracker.
- reports.
- export/import.

34.6 WS-F - Discovery and career intelligence

Tasks:

- matching.
- company normalization.
- fraud signals.

- networking.
- outcome learning.

34.7 WS-G - Packaging and launch

Tasks:

- PyPI.
- Docker.
- desktop builds.
- updater/signing.
- docs site.
- release runbook.

Dependencies must be enforced by the task graph, not informal coordination.

35 33. High-risk implementation items requiring special handling

35.1 LinkedIn

High platform/ToS risk. Use assisted/manual workflows unless the exact automation is demonstrably authorized. Do not build detection bypass mechanisms.

35.2 Direct ATS APIs

Do not assume a public read API implies public applicant submission. Verify authentication, permissions and terms per integration.

35.3 Credential-bearing tools

Any tool with access to email, browser sessions, credentials or local secrets needs explicit permission scopes.

35.4 Generated candidate content

Every external candidate claim requires candidate-truth grounding.

35.5 Self-improvement

No direct production mutation. All changes go through sandbox + eval + baseline comparison.

35.6 Telemetry

No default data egress. Opt-in only. Redaction tests are release gates.

35.7 Multi-worker execution

Do not scale worker count until single-worker correctness is demonstrated under crash/restart conditions.

36 34. Documentation and artifact checklist for the final implementation agent

At completion, the agent must ensure these files exist or are intentionally omitted with a documented reason:

```
README.md
CHANGELOG.md
SECURITY.md
CONTRIBUTING.md
CODE_OF_CONDUCT.md
LICENSE
CODEOWNERS
```

```
SETUP.md
```

```
docs/README.md
docs/architecture.md
docs/contracts.md
docs/security.md
docs/privacy.md
docs/release-policy.md
docs/release-runbook.md
docs/cli-reference.md
docs/api-reference.md
docs/gui-guide.md
docs/adapters.md
docs/memory.md
docs/evaluation.md
docs/troubleshooting.md
docs/faq.md
docs/migration.md
docs/development.md
```

```
docs/implementation/master-plan.md
```

docs/implementation/requirements-matrix.md
docs/implementation/source-reconciliation.md

queues/now.md
queues/next.md
queues/blocked.md
queues/improve.md
queues/recurring.md
state/project.md
state/plan.md
state/tasks.md
state/knowledge.md
state/decisions.md
state/status.md
state/handoff.md
state/FAILURE.md

37 35. Final execution sequence

The entire program should be executed in this dependency order:

0. Baseline + source reconciliation
|
1. Security + supply-chain blockers
|
2. Durable tasks + events + leases
|
3. Application state machine + effects + approvals
|
4. Browser reliability + adapter contracts
|
5. LLM streaming + routing + candidate truth + memory
|
6. Documents + matching + evidence
|
7. GUI/control plane + observability
|
8. Packaging + versioning + migrations + backup
|
9. Evaluation + failure injection + performance hardening
|
10. MCP/API/plugins + authorized integrations
|
11. Product differentiation + outcome learning

- |
- 12. Bounded self-improvement + skill extraction
- |
- 13. Release candidate
- |
- 14. v1.0.0 stable

The implementation agent may reorder work within a phase when dependency analysis proves it is safe, but must not skip a phase's exit criterion merely to accelerate feature output.

38 36. Recommended release sequence

- 0.x experimental
- > 0.9 release candidate
- > 1.0 stable supported workflow
- > 1.0.x reliability/adapters
- > 1.1 product expansion
- > 1.2 interoperability
- > 2.x broader career-OS capabilities

The first stable release should be intentionally narrower and more reliable than an unchecked claim of universal automation.

39 37. Post-launch operating model

39.1 Weekly

- issue triage.
- adapter health review.
- incident review.
- security advisory review.
- eval regression review.
- queue reconciliation.

39.2 Monthly

- dependency updates.
- release train.
- performance review.
- cost review.
- memory/data retention review.
- outcome-learning review.

39.3 Per release

- new baseline.
- security scan.
- full eval.
- migration test.
- packaging verification.
- changelog.
- docs build.
- release dossier.

39.4 Continuous

- monitor external standards.
 - inspect important upstream changes.
 - run bounded experiments.
 - retain only improvements supported by evidence.
-

40 38. Final design principles

The completed JoBot should embody these principles:

1. **Durability before autonomy.**
 2. **Verification before confidence.**
 3. **Evidence before claims.**
 4. **Truth before persuasion.**
 5. **Explicit state before hidden memory.**
 6. **Human governance before irreversible action.**
 7. **Deterministic rails with AI at the edges.**
 8. **Adapters are replaceable; domain state is not.**
 9. **Local-first by default; external data egress by consent.**
 10. **Performance is cost per successful verified outcome, not raw speed.**
 11. **Every improvement must survive evaluation.**
 12. **A failure is valuable only when it becomes visible, diagnosable and actionable.**
 13. **The safest path to a general agent is a trustworthy specialized harness.**
 14. **The goal is not maximum applications; it is maximum expected career value per unit of user attention, cost and risk.**
-

41 39. External research basis

The implementation agent should use primary sources and current documentation whenever a dependency, external API, protocol or platform rule changes.

Current research checked while preparing this synthesis includes:

- Model Context Protocol 2026 specification update and transport documentation: <https://blog.modelcontextprotocol.io/posts/2026-07-28/> and <https://modelcontextprotocol.io/specification/draft/basic/transport>
- Greenhouse Job Board API: <https://developers.greenhouse.io/job-board>
- Ashby application form API: <https://developers.ashbyhq.com/reference/applicationformsubmit-1>
- Playwright locators/actionability: <https://playwright.dev/docs/locators> and <https://playwright.dev/docs/actionability>
- OpenTelemetry documentation: <https://opentelemetry.io/docs/>
- PyPI Trusted Publishing: <https://docs.pypi.org/trusted-publishers/>
- Tauri 2 capabilities/CSP: <https://v2.tauri.app/security/capabilities/> and <https://v2.tauri.app/security/csp/>
- LinkedIn User Agreement: <https://www.linkedin.com/legal/user-agreement>
- Current market-facing product capability references: Simplify, Teal, Jobright, Huntr official documentation/sites

External information must not be copied into the system as a permanent assumption. Re-check current official documentation when implementing the corresponding integration.

42 40. Master completion rule

The agent implementing this plan should consider the work complete only when it can answer **yes** to all of the following, with evidence:

- Can JoBot survive a process kill and resume a long-running task?
- Can it prove it did not duplicate an external side effect?
- Can it distinguish verified success from unknown state?
- Can a human inspect and control consequential actions?
- Can candidate claims be traced to evidence?
- Can every LLM call be budgeted and correlated with a task?
- Can memory be explained and deleted?
- Can plugins be permissioned and isolated?
- Can the system diagnose itself with jobot doctor?
- Can the repository be installed and run by an average user?
- Can CI reproduce release artifacts?
- Can the release be upgraded and recovered safely?
- Can the GUI show what the agent is doing without hiding critical state?

Can failure be injected and recovered from deterministically?
Can performance and resource use be measured rather than guessed?
Can the project documentation be trusted because it is generated/verified against reality?
Can the product expand without weakening the durable execution core?

Only when the answer is yes, backed by release evidence, should the project be described as production-ready.